

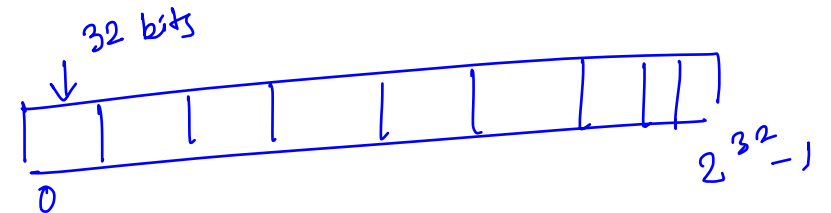
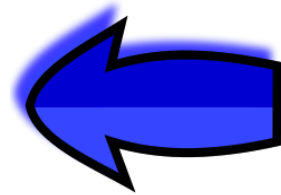


Chapter 11: The Memory System

Basic Computer Architecture

Outline

- * Overview of the Memory System
- * Caches
- * Details of the Memory System
- * Virtual Memory



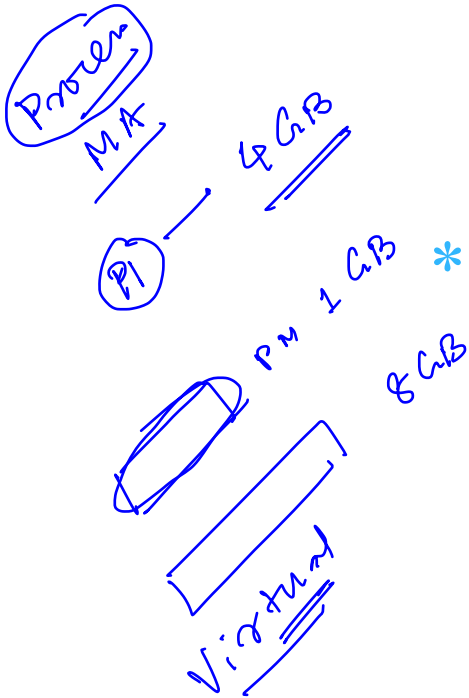
Need for a Fast Memory System

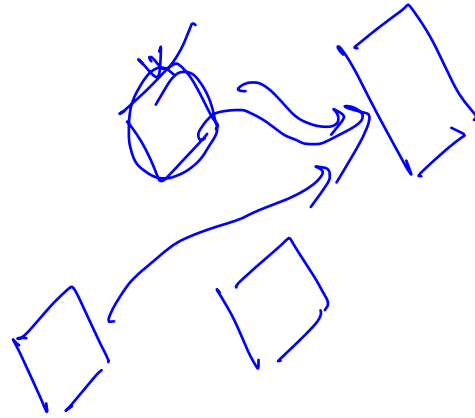
- * We have up till now **assumed** that the **memory** is one large array of bytes

- * Starts at 0, and ends at $(2^{32} - 1)$
- * Takes **1 cycle** to access memory (read/write)

- * All programs share the memory

- * We somehow **magically** avoid overlaps between programs running on the same processor
- * All our **programs** require less than 4 GB of space





Windows Task Manager

File Options View Help

Applications Processes Services Performance Networking Users

Image Name	User Name	CPU	Memory (...)	Description
Skype.exe *32	Dell	00	89,000 K	Skype
POWERPNT.EXE	Dell	00	79,896 K	Microsoft ...
GingerClient.e...	Dell	00	62,584 K	Ginger
explorer.exe	Dell	00	25,300 K	Windows ...
nis.exe *32	Dell	00	12,608 K	Norton In...
dwm.exe	Dell	00	11,744 K	Desktop ...
TortoiseHgOv...	Dell	00	11,260 K	TortoiseH...
PrivacyIconCli...	Dell	00	9,204 K	Intel(R) M...
GingerService...	Dell	00	8,376 K	Ginger
RAVBg64.exe	Dell	00	6,024 K	HD Audio ...
RAVBg64.exe		00	4,896 K	
RtkNGUI64.exe	Dell	00	4,096 K	Realtek H...
taskhost.exe	Dell	00	3,928 K	Host Proc...
acrotray.exe ...	Dell	00	3,616 K	AcroTray
igfxEM.exe	Dell	00	3,424 K	igfxEM M...
WzPreloader....	Dell	00	3,396 K	WinZip Pr...
Apoint.exe	Dell	00	3,356 K	Alps Point...
igfxTray.exe	Dell	00	3,160 K	igfxTray
taskmgr.exe	Dell	00	2,888 K	Windows ...

Show processes from all users End Process

Processes 74 CPU Usage: 0% Physical Memory: 42%

All the programs running on my machine. The CPU of course runs one program at a time. Switches between programs periodically.

Regarding all the memory being homogeneous → NOT TRUE

Cell Type	Area	Typical Latency
Master Slave D flip flop	$0.8 \mu m^2$	Fraction of a cycle
SRAM cell in an array	$0.08 \mu m^2$	1-5 cycles
DRAM cell in an array	$0.005 \mu m^2$	50-200 cycles

Typical Values

area ↓ latency ↑
Power ↓ latency ↑

* Should we make our memory using only flip-flops ?

- * 10X the area of a memory with SRAM cells
- * 160X the area of a memory with DRAM cells
- * Significantly more power !!!



Tradeoffs

- * Tradeoffs

- * Area, Power, and Latency

- * Increase Area → Reduce latency, increase power

- * Reduce latency → increase area, increase power

- * Reduce power → reduce area, increase latency

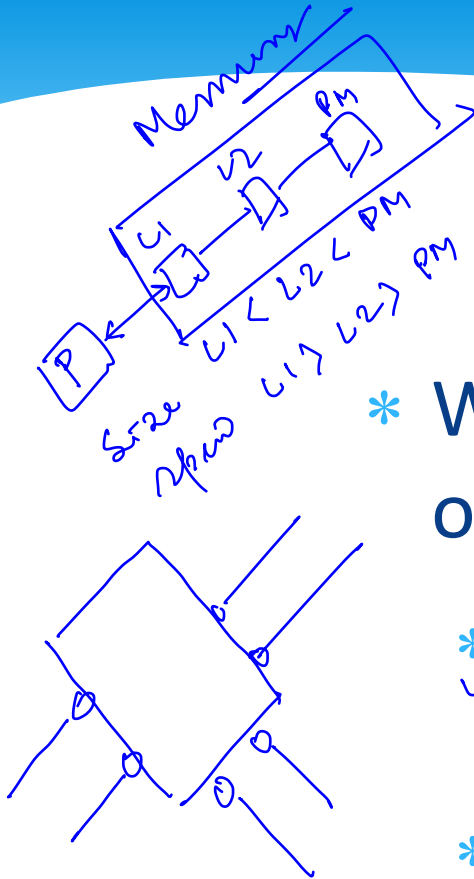
- * We cannot have
the best of all worlds



What do we do ?

- * We cannot create a memory of just flip flops
 - * We will hardly be able to store anything
- * We cannot create a memory of just SRAM cells
 - * We need more storage, and we will not have a 1 cycle latency
- * We cannot create a memory of DRAM cells
 - * We cannot afford 50+ cycles per access

Memory Access Latency



* What does memory access latency depend on ?

* Size of the memory → larger is the size, slower it is

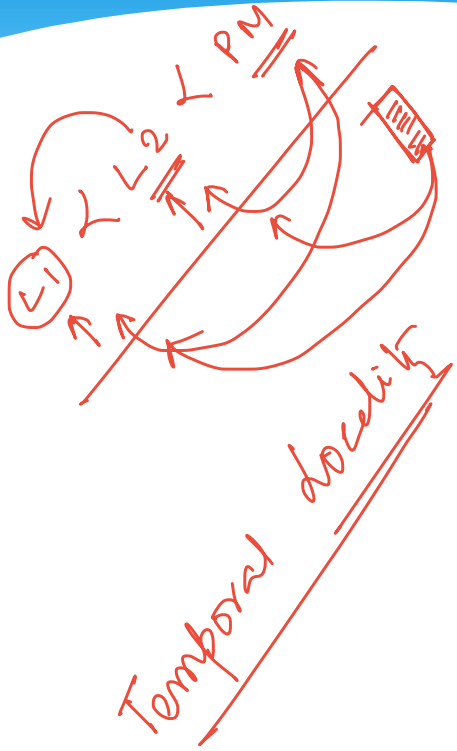
* **Number of ports** → More are the ports (parallel accesses/cycle), slower is the memory

* **Technology** used → SRAM, DRAM, flip-flops

$a[100]$
 $a[10]$

hit / miss

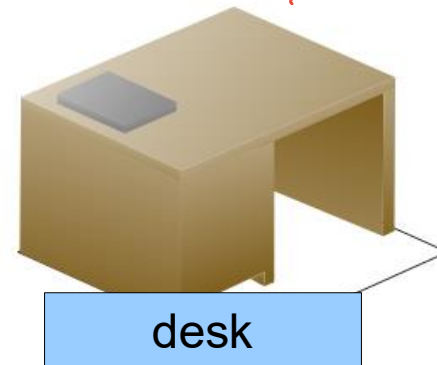
Solution : Leverage Patterns



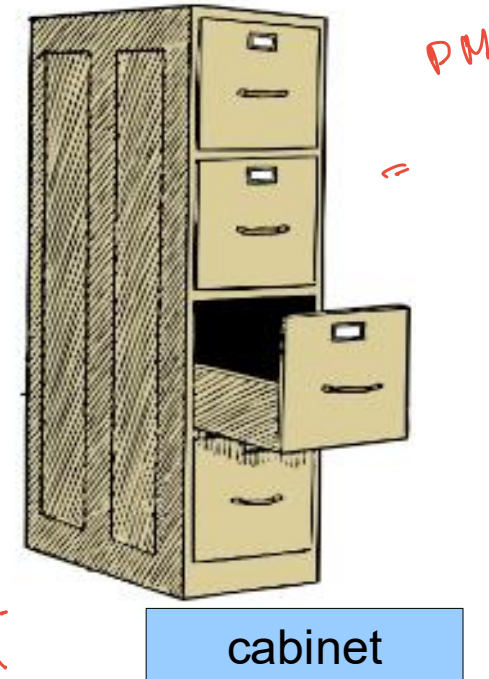
- * Look at an example in real life
 - * Sofia's workplace



= L2



= L1



A Protocol with Books

- * Sofia keeps the
 - * **most frequently** accessed books on her desk
 - * **slightly less frequently** accessed books on the shelf
 - * **rarely accessed** books in the cabinet
- * Why ?
 - * She tends to read the same set of books over and over again, in the same window of time → **Temporal Locality**

Protocol – II

- * If Sofia takes a **computer architecture** course
 - * She has comp. architecture books on her **desk**
- * After the course is over
 - * The **architecture** books go back to the **shelf**
 - * And, **vacation planning** books come to the desk
 - * **Idea** : Bring all the **vacation planning** books in one go. If she requires one, in high likelihood she might require similar books in the near future.



Temporal and Spatial Locality

Temporal Locality

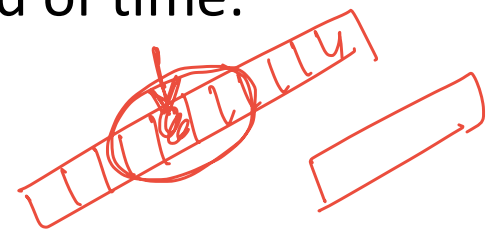
It is a concept that states that if a resource is accessed at some point of time, then most likely it will be accessed again in a short period of time.

Spatial Locality

It is a concept that states that if a resource is accessed at some point of time, then most likely similar resources will be accessed again in the near future.

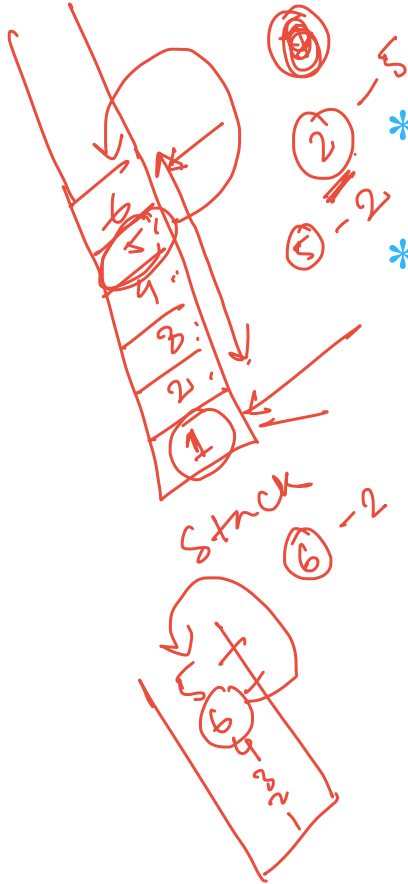
5

1
2
3
4



1000 ← 500 300 200

Temporal Locality in Programs

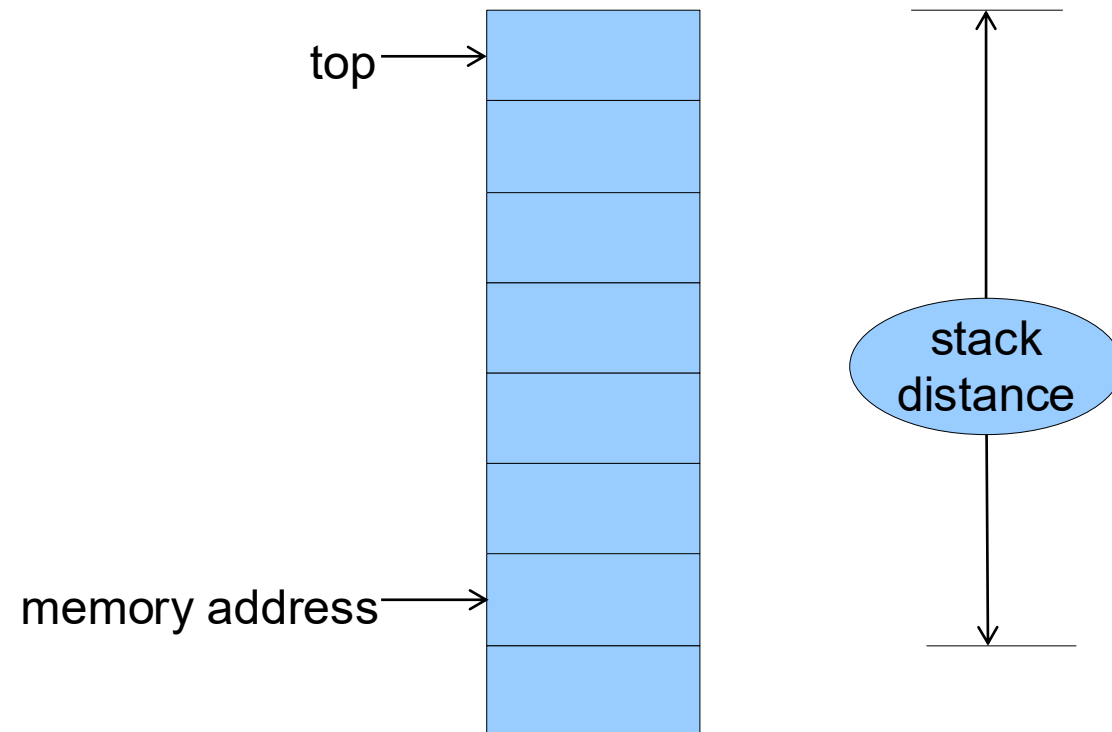


* Let us verify if programs have temporal locality

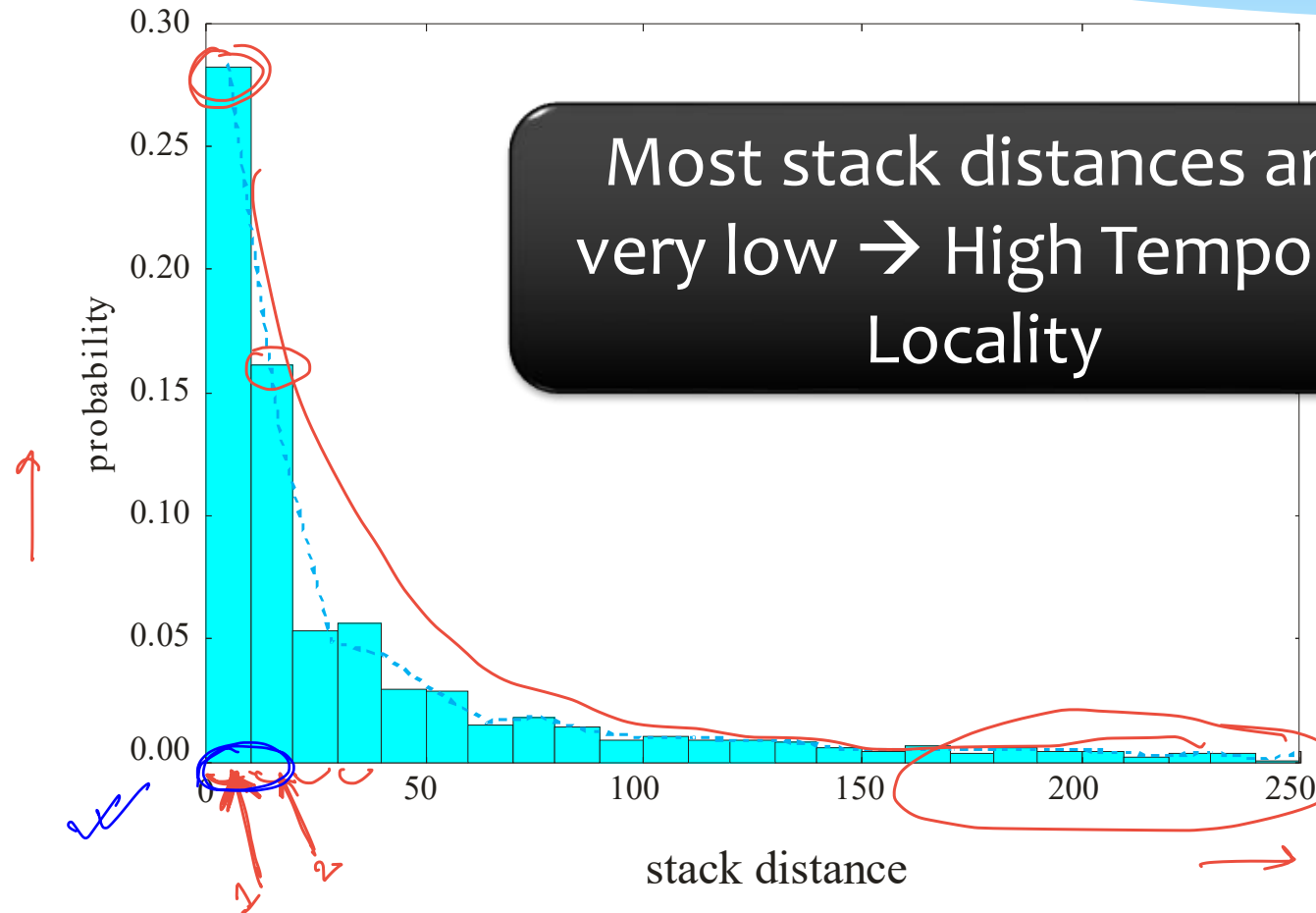
* **Stack distance**

- * Have a **stack** to store memory addresses.
- * Whenever, we **access an address** → we bring it to the top of the stack
- * Stack distance → Distance from the top of the **stack** to where the element was found
- * Quantifies reuse of addresses

Stack Distance



Stack Distance Distribution



* Benchmark : Set of perl programs

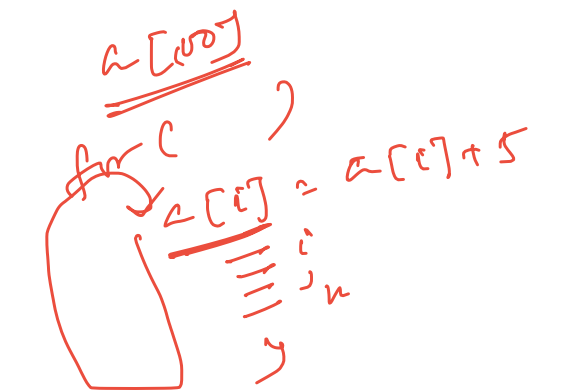
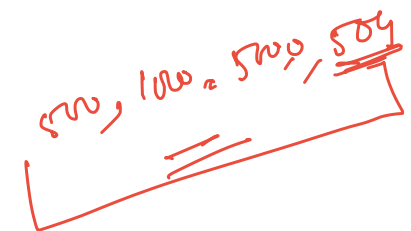
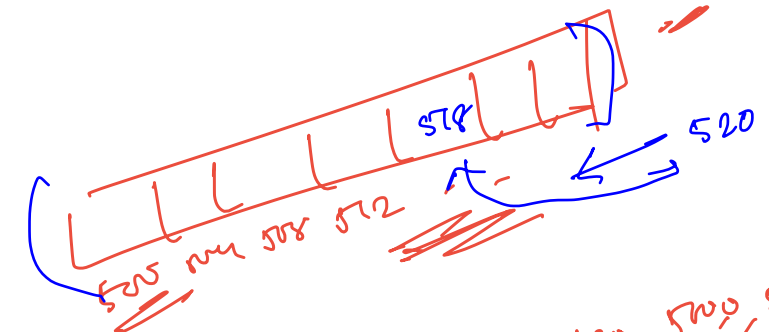
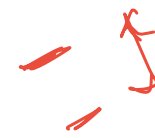
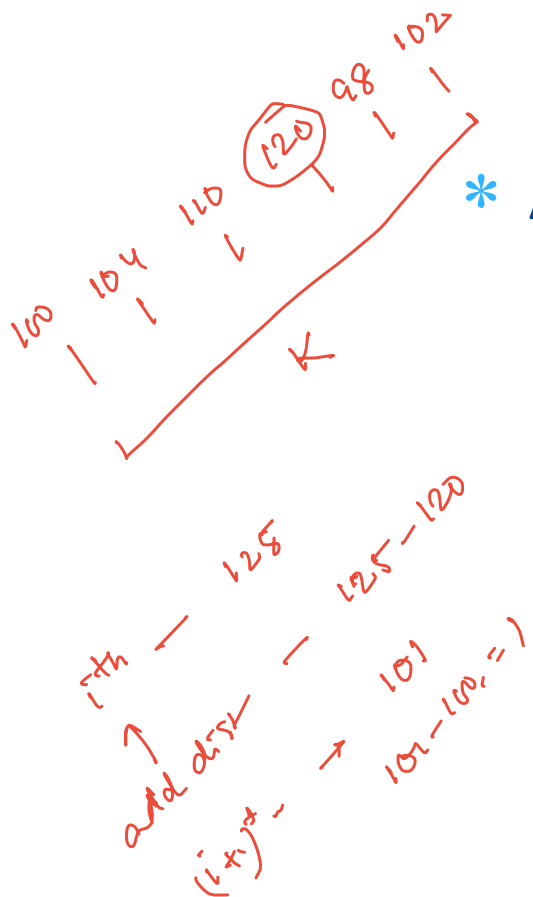
Address Distance

- * Maintain a **sliding window** of the last K memory accesses

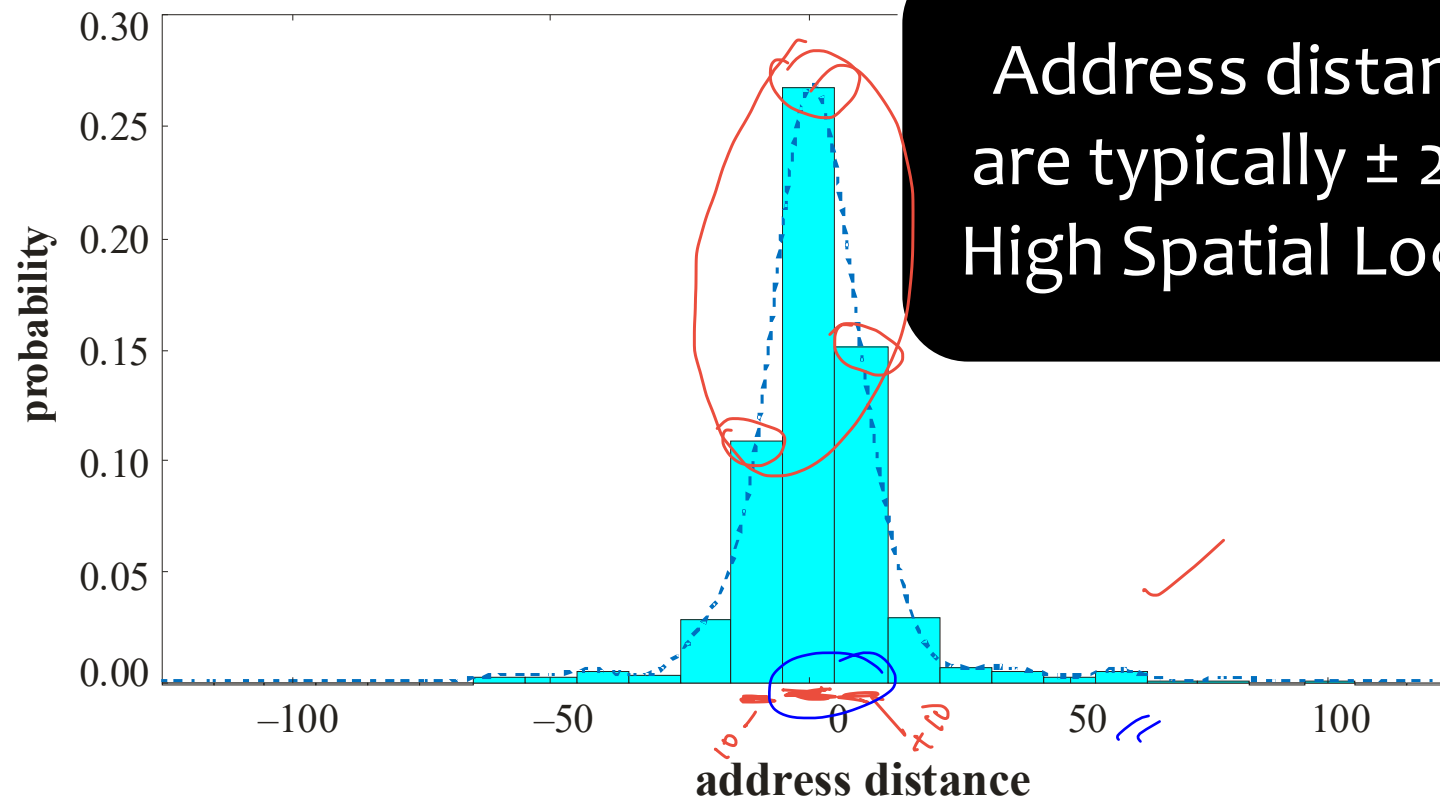
- * Address distance :

- * The i^{th} address distance is the difference in the memory addresses of the i^{th} memory access, and the closest address in the set of last K memory accesses.

- * Shows the **similarity** in addresses



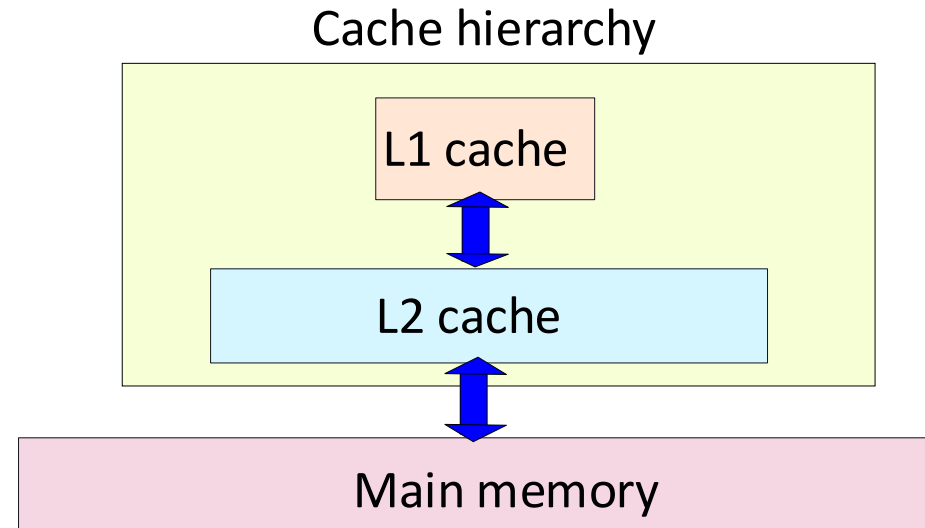
Address Distance Distribution



Address distances
are typically $\pm 20 \rightarrow$
High Spatial Locality

K=10, benchmark consisting of *perl* programs

Exploiting Temporal Locality



L1 C L2 C L3 C

- * Use a **hierarchical memory** system
- * L1 (SRAM cells), L2 (SRAM cells), Main Memory (DRAM cells)

The Caches

- * The **L1 cache** is a small memory (8-64 KB) composed of **SRAM cells**
- * The **L2 cache** is larger and slower (128 KB – 4 MB) (**SRAM cells**)
- * The **main memory** is even larger (1 – 64 GB) (**DRAM cells**)
- * **Cache hierarchy**
 - * The **main memory** contains all the memory locations
 - * The **caches** contain a subset of memory locations

Access Protocol

- * Inclusive Cache Hierarchy

- * $\text{addresses}(L1) \sqsubset \text{addresses}(L2) \sqsubset \text{addresses}(\text{main memory})$

- * Protocol

- * First access the L1 cache. If the memory location is present, we have a **cache hit**.
 - * Perform the access (read/write)
 - * Otherwise, we have a **cache miss**.
 - * Fetch the value from the lower levels of the memory system, and populate the cache.
 - * Follow this protocol recursively

Advantage

- * Typical Hit Rates, Latencies

- * L1 : 95 %, 1 cycle

- * L2 : 60 %, 10 cycles

- * Main Memory : 100 %, 300 cycles

- * Result :

- * 95 % of the memory accesses take a single cycle

- * 3 % take, 10 cycles

- * 2 % take, 300 cycles

Exploiting Spatial Locality

- * Conclusion from the address locality plot

- * Most of the addresses are within +/- 25 bytes



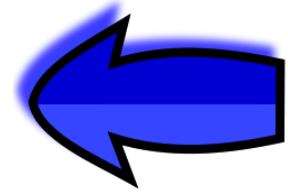
- * Idea :

- * Group memory addresses into sets of n bytes
 - * Each group is known as a cache line or cache block
 - * A cache block is typically 32, 64, or 128 bytes

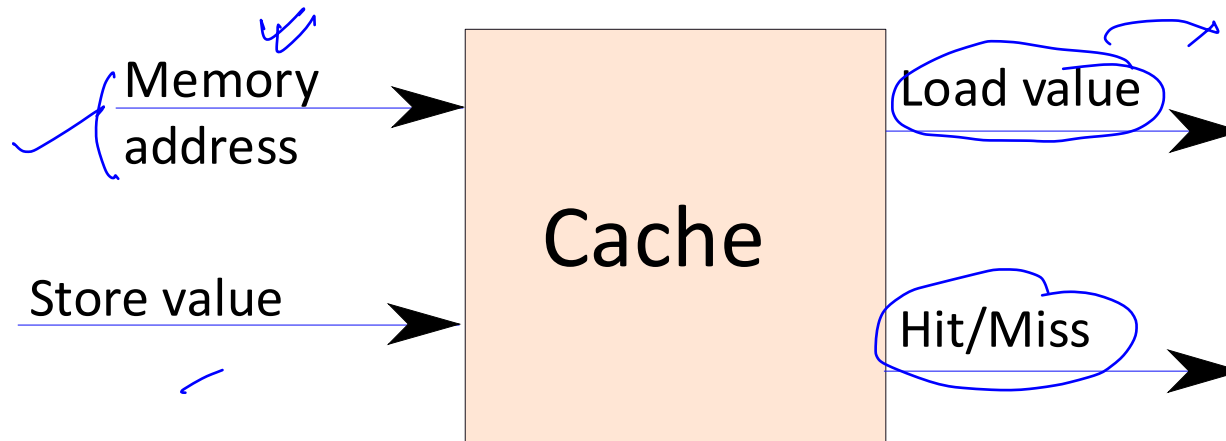
Reason: Once we fetch a block of 32/64 bytes. A lot of accesses in a short time interval will find their data in the block.

Outline

- * Overview of the Memory System
- * Caches
- * Details of the Memory System
- * Virtual Memory



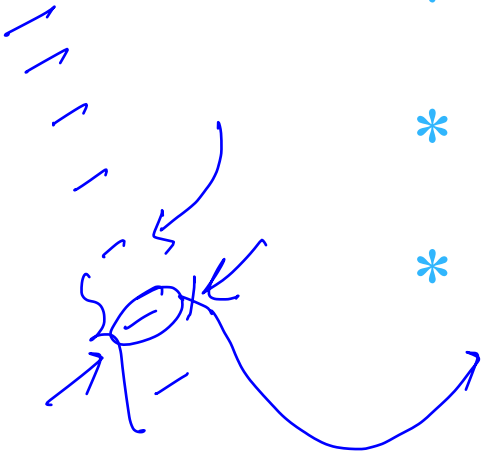
Overview of a Basic Cache



- * Saves a subset of memory values
 - * We can either have hit or miss
 - * The load/store is successful if we have a hit

Basic Cache Operations

- * **lookup** → Check if the memory location is present
- * **data read** → read data from the cache
- * **data write** → write data to the cache
- * **insert** → insert a block into a cache
- * **replace** → find a candidate for replacement
- * **evict** → throw a block out of the cache



Cache Lookup

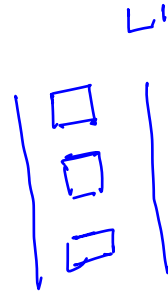
B - Byte
b - bit

- * Running example : 8 KB Cache, block size of 64 bytes, 32 bit memory system

(L1)
 $8KB = 2^3 \times 2^{10}$
 $= 2^{13}B$

$$\frac{2^{32}}{2^6} = 2^{26}$$

- * Let us have two SRAM arrays



- * tag array → Saves a part of the block address such that the block can be uniquely identified
- * block array → Saves the contents of the block
- * Both the arrays have the same number of entries

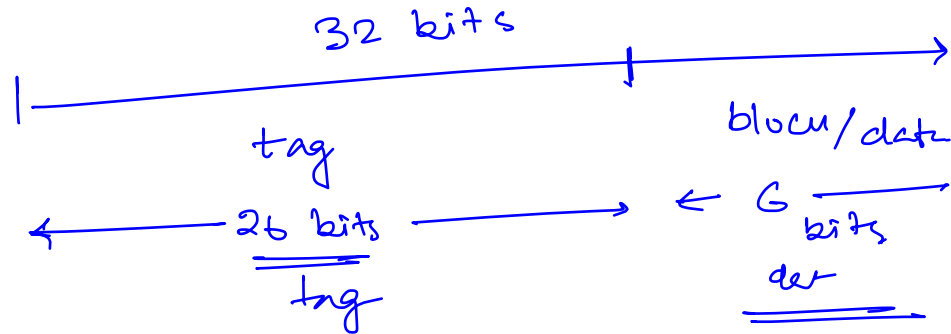
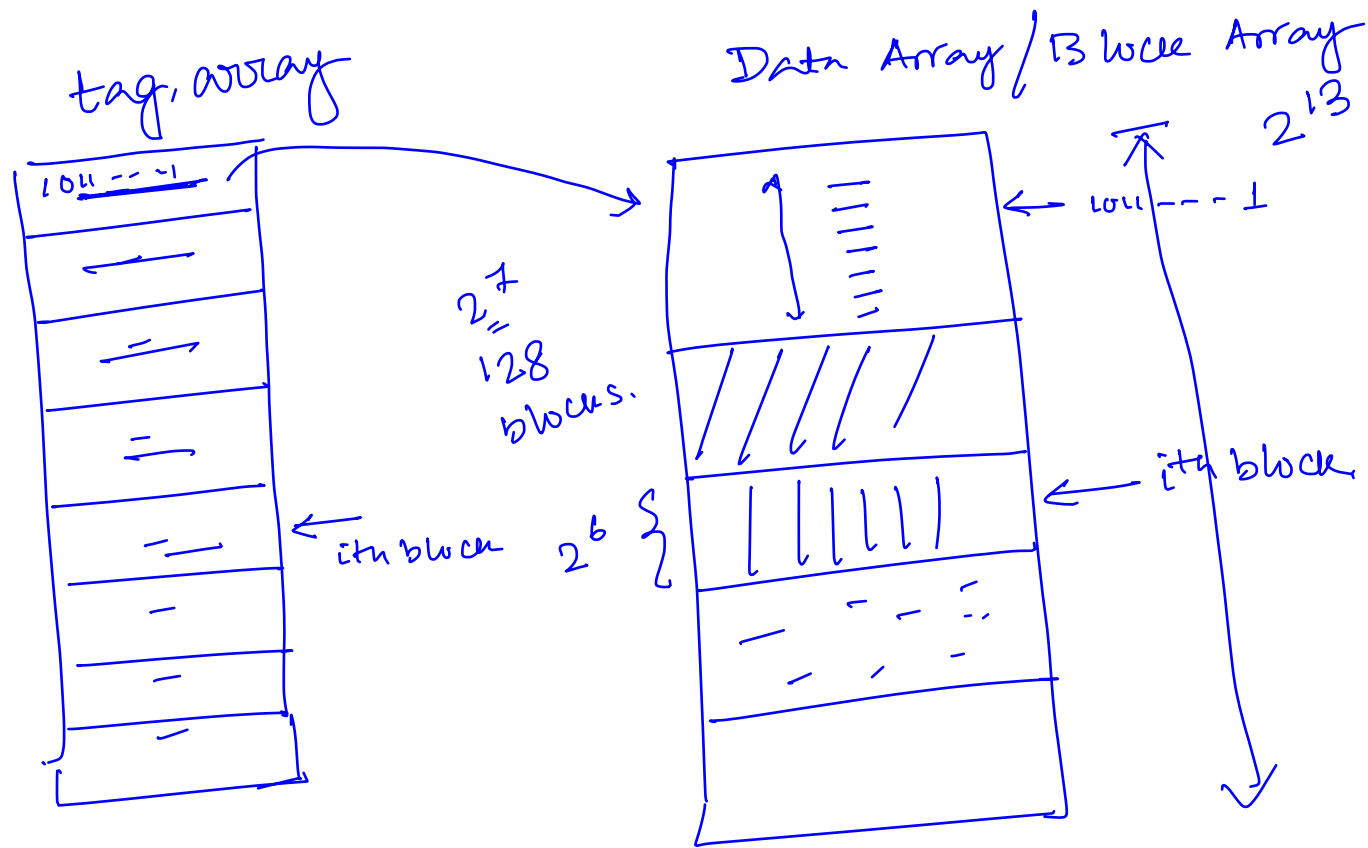
Block = 64B = $\frac{2^{13}}{2^6} = 2^7$

L1
.....

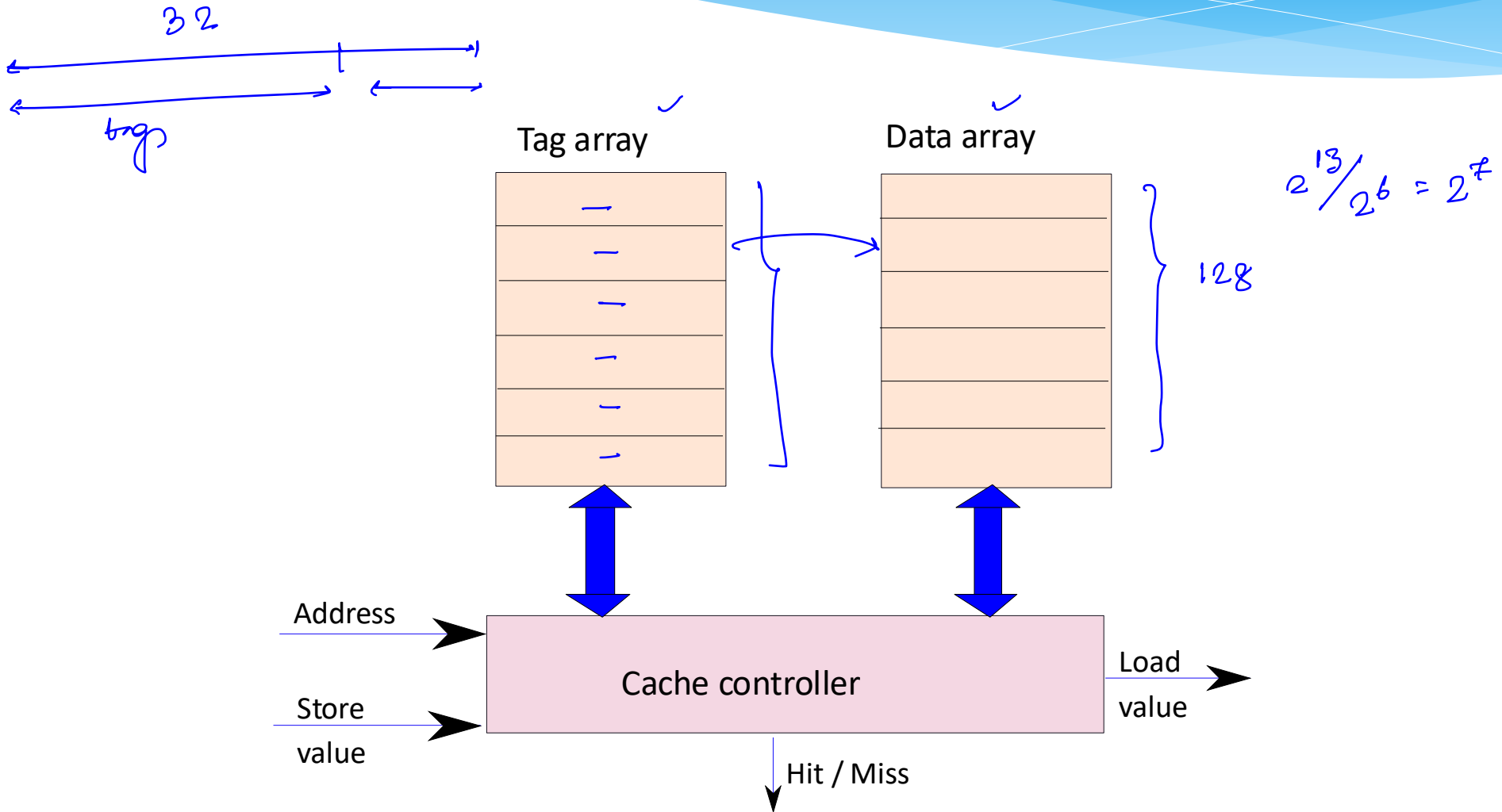
Atomic unit - block

128
cells

$$\frac{2^{13}}{2^6} = 2^7 \text{ entries.} \\ = 128$$

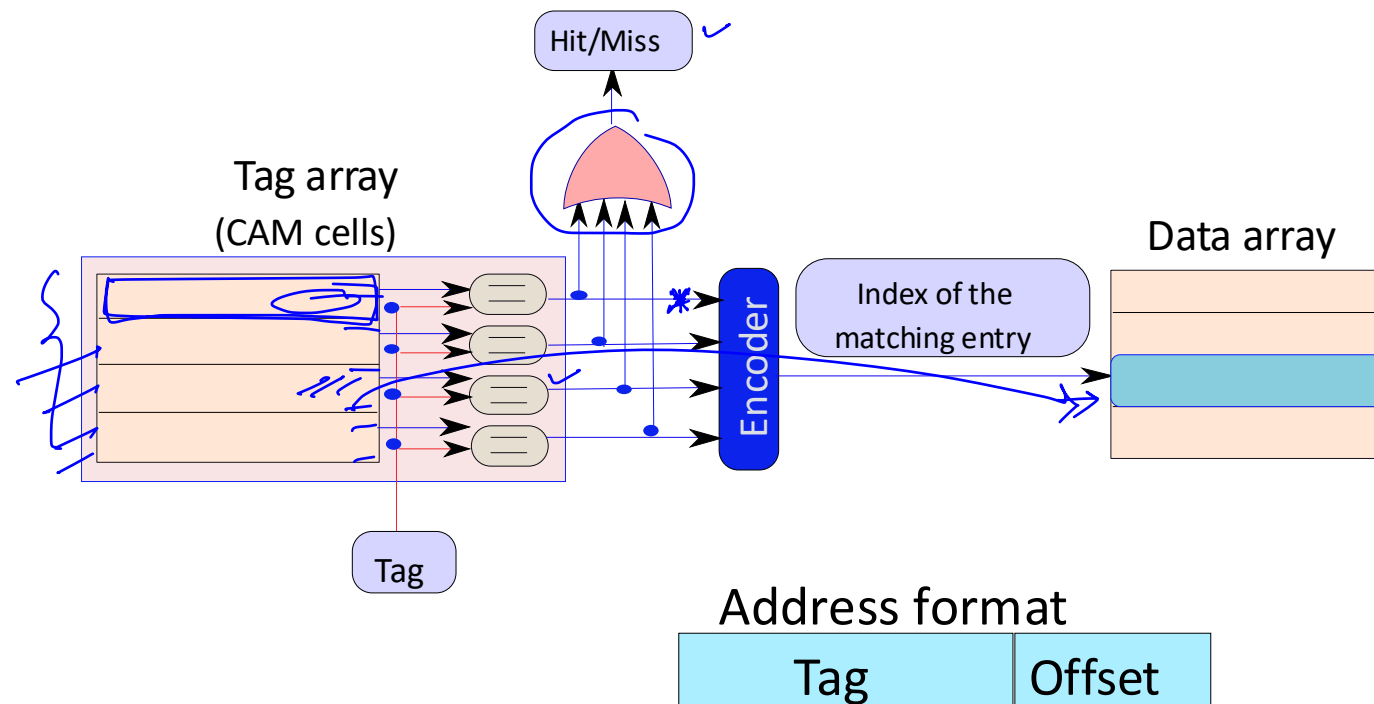


Structure of a Cache



Fully Associative Cache

- * We have $2^{13} / 2^6 = 128$ entries
- * A block can be saved in any entry
- * 26 bit tag, and 6 bit offset

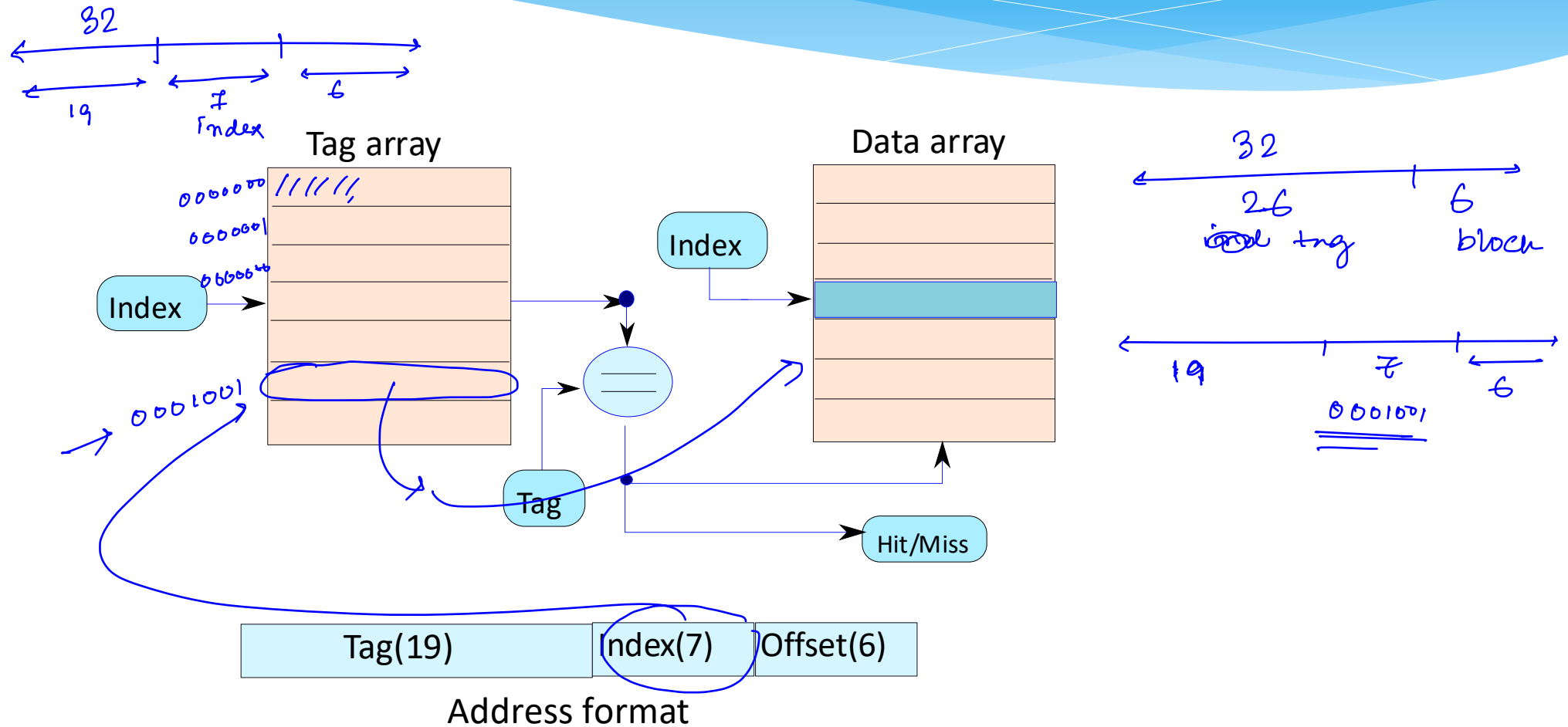


Implementation of the FA Cache

- * We use an array of CAM cells for the tag array
- * Each entry compares its contents with the tag
- * Sets the match line to 1
- * The OR gate computes a hit or miss
- * The encoder computes the index of the matching entry.
- * We then read the contents of the matching entry from the block array

Refer to Chapter 6: Digital Logic

Direct Mapped Cache

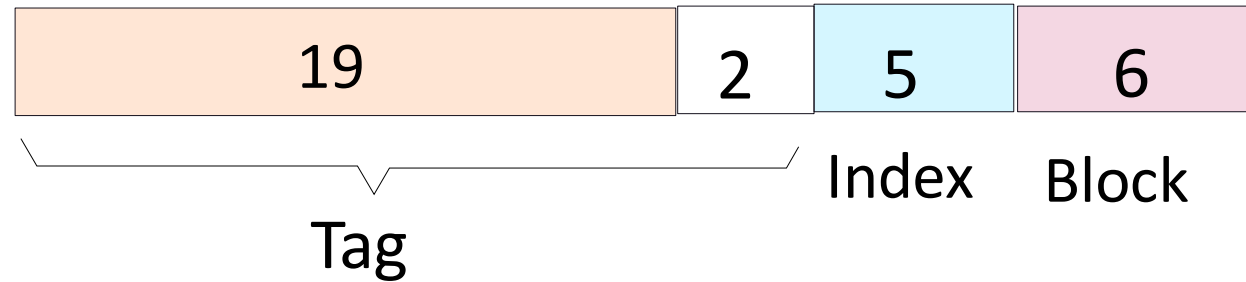


- * Each block can be mapped to only 1 entry

Direct Mapped Cache

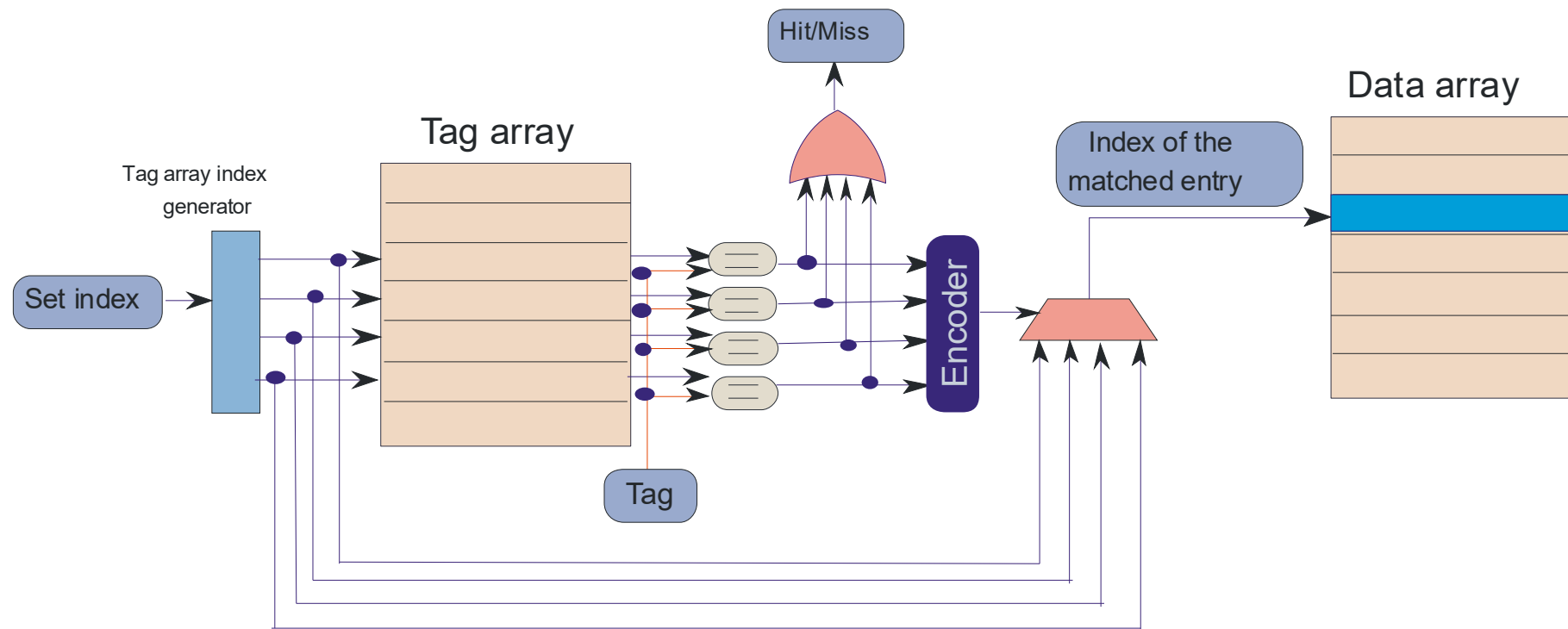
- * We have 128 entries in our **cache**.
- * We compute the **index** as $\text{idx} = \text{block address} \% 128$
- * We access **entry**, **idx**, in the **tag array** and compare the contents of the **tag** (19 msb bits of the **address**)
- * If there is a **match** → **hit**
 - * else → **miss**
- * Need a solution that is in the middle of the spectrum

Set Associative Cache



- * Let us assume that an address can reside in 4 locations
- * Access all 4 locations, and see if there is a hit
- * Thus, we have $128/4 = 32$ indices
- * Each index points to a set of 4 entries
- * → We now use a 21 bit tag, 5 bit index

Set Associative Cache



Set Associative Cache

- * Let the **index** be i , and the **number of elements** in a set be k
 - * We access **indices**, $i*k$, $i*k+1$, ..., $i*k + (k-1)$
 - * Read all the **tags** in the set
 - * **Compare** the tags with the tag obtained from the address
 - * Use an OR gate to compute a **hit/ miss**
 - * Use an **encoder** to find the **index** of the **matched entry**

Set Associative Cache – II

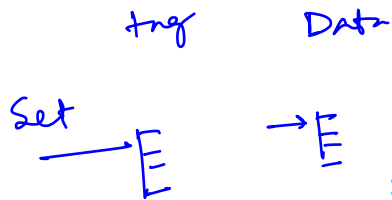
- * Read the corresponding entry from the block array
- * Each entry in a set is known as a way
- * A cache with k blocks in a set is known as a k-way associative cache

hit rate: $FA \rightarrow SA \rightarrow DM$
search time: $DM \rightarrow SA \rightarrow FA$
Fast

Data read operation

load

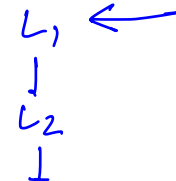
- * This is a **regular** SRAM access.
- * Note that the **data read** and **lookup** can be overlapped for a **load access**
- * We can issue a parallel data read to all the ways in the cache



- * Once, we compute the index of the **matching** tag, we can choose the **correct** result with a **multiplexer**.

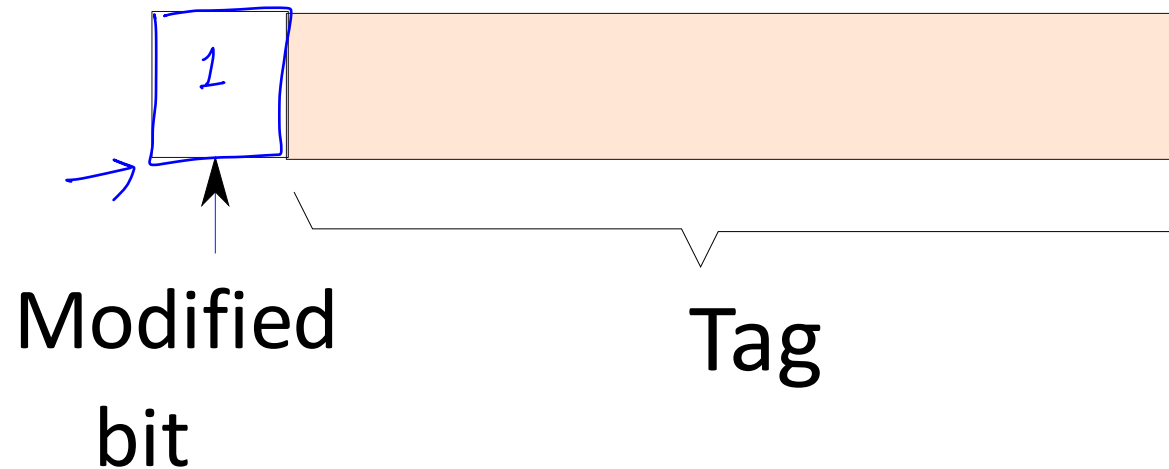
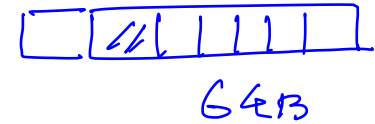
Data write operation

- * Before we write a value
 - * We need to ensure that the block is present in the cache
- * Why ?
 - * Otherwise, we have to maintain the indices of the bytes that were written to
 - * We treat a block as an atomic unit
 - * Hence, on a miss, we fetch the entire block first
- * Once a block is there in the cache
 - * Go ahead and write to it



Modified bit

- * Maintain a **modified** bit in the tag array.
- * If a **block** has been written to, after it was **fetch**ed, set it to 1.



Write Policies



* **Write through** → Whenever we write to a cache, we also write to its lower level

* Advantage : Can **seamlessly evict** data from the cache

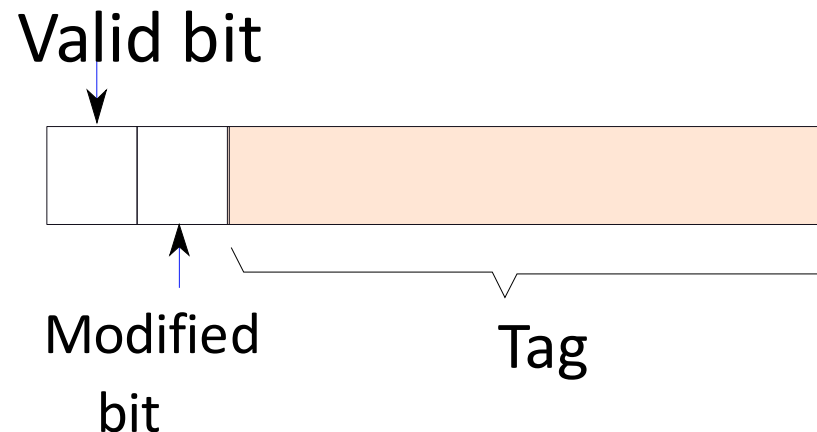
* **Write back** → We do not **write** to the lower level. Whenever we write, we set the **modified bit**.

* At the time of **eviction** of the line, we check the value of the **modified bit**

insert operation

If we don't find a block in a cache. We fetch it from the lower level. Then we insert the block in the cache → insert operation

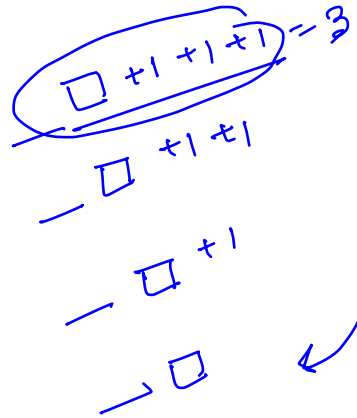
- * Let us add a **valid** bit to a **tag**
 - * If the line is non-empty, valid bit is 1
 - * Else it is 0
- * Structure of a tag



insert operation - II

- * Check if any way in a set has an **invalid** line
 - * If there is one, then write the **fetch** line to that **location**, set the **valid** bit to 1.
- * Otherwise,
 - * find a candidate for **replacement**

The replace operation



* A cache replacement scheme or **replacement** policy is a method to **replace** an entry in the set by a new entry

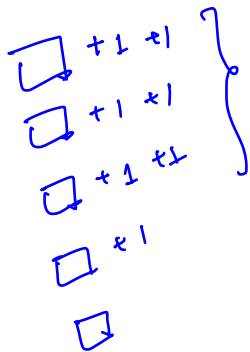
* Replacement Schemes

- * Random replacement scheme

- * FIFO replacement scheme

- * When we fetch a block, assign it a **counter value** equal to 0

- * Increment the **counters** of the rest of the **ways**



Replacement Schemes

- * FIFO

- * For replacement, choose the way with the highest counter (oldest).

- * Problems :

- * Can **violate** the principle of temporal locality
 - * A line fetched early might be accessed very frequently.

LRU (least recently used)

- * **Replace** the **block** that has been accessed the least in the **recent past**
 - * Most likely we will not access it in the near future
 - * Directly follows from the definition of **stack distance**
 - * Sadly, we need to do more **work** per access
 - * Proved to be **optimal** in some restrictive scenarios
- * **True** LRU requires saving a hefty **timestamp** with every way
 - * Let us implement pseudo-LRU

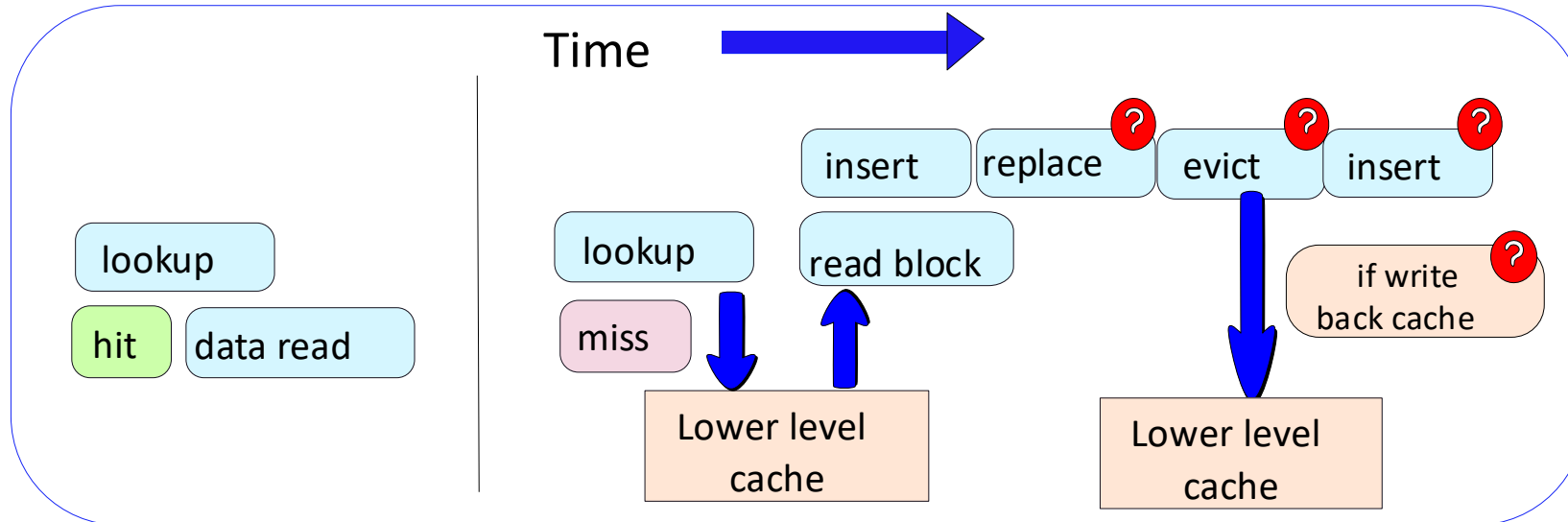
Pseudo-LRU

- * Let us try to **mark** the **most recently used** (MRU) elements.
- * Let us **associate** a 3 bit counter with every **way**.
 - * Whenever we **access** a **line**, we **increment** the counter.
 - * We stop **incrementing** beyond 7.
 - * We periodically **decrement** all the **counters** in a set by 1.
 - * Set the counter to 7 for a newly **fetch**ed block
 - * For **replacement**, choose the **block** with the **smallest** counter.

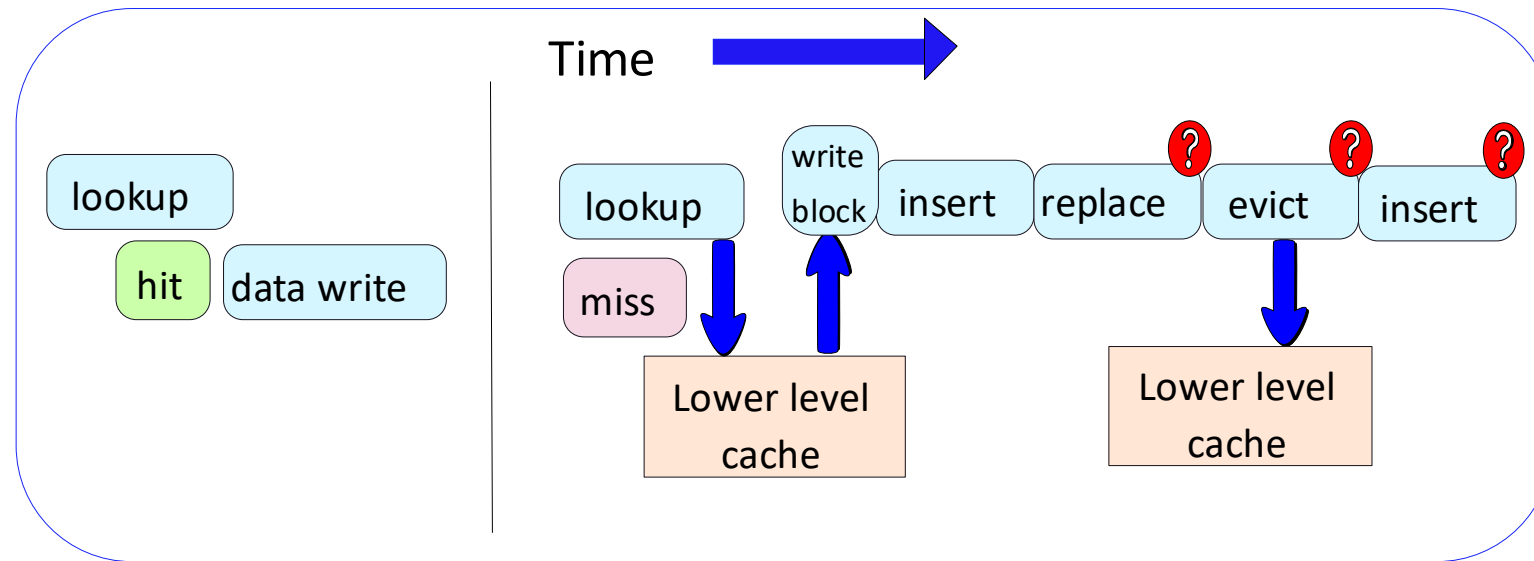
evict Operation

- * If the cache is write-through
 - * Nothing needs to be done
- * If the cache is write-back
 - * AND the modified bit is 1
 - * Write the line to the lower level

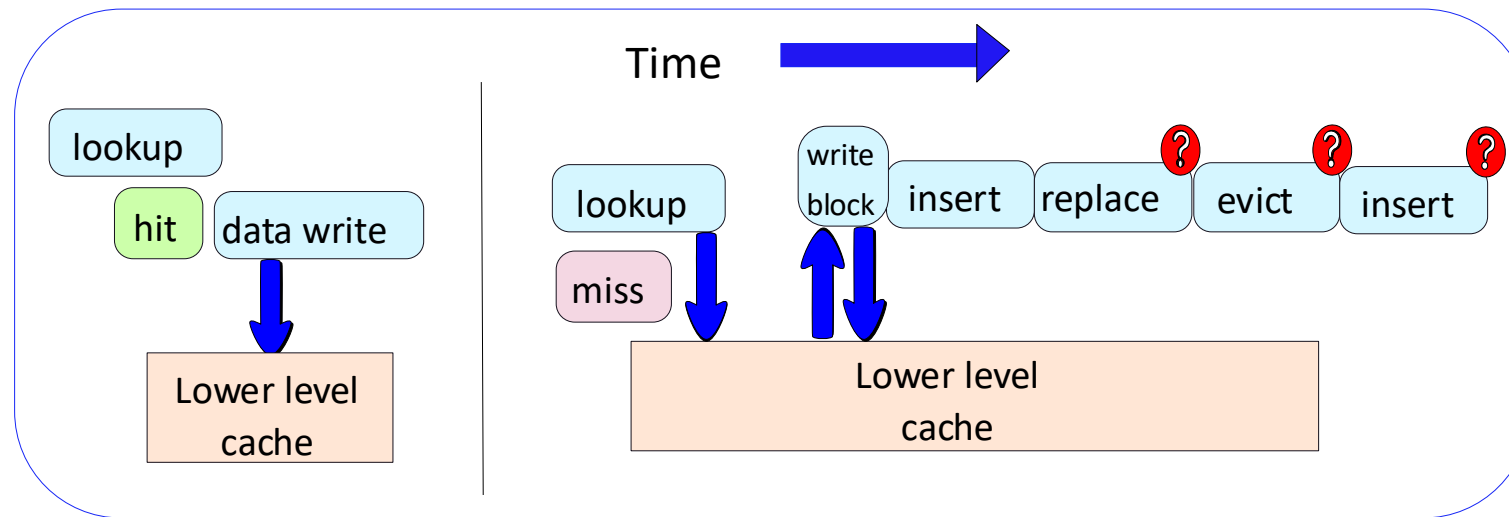
The read(load) Operation



Write operation in a write back cache

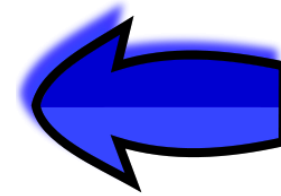


Write operation in a write through cache



Outline

- * Overview of the Memory System
- * Caches
- * Details of the Memory System
- * Virtual Memory



Mathematical Model of the Memory System

$$\begin{aligned} \underline{CPI} &= \underline{CPI}_{ideal} + \text{stall}_{rate} * \text{stall}_{cycles} \\ &= \underline{CPI}_{ideal} + f_{mem} * (\underbrace{AMAT}_{1.4} - \underbrace{1}_{1}) \end{aligned}$$

- * **AMAT** $\rightarrow$ Average Memory Access Time
- * f_{mem} $\rightarrow$ Fraction of memory instructions
- * **CPI_{ideal}** $\rightarrow$ ideal CPI assuming a perfect 1 cycle memory system

$$\begin{aligned} \text{Penalty} \\ &= AMAT - \text{L1 hit time} \end{aligned}$$

$$AMAT = L1_{hit\ time} + L1_{miss\ rate} \times L1_{miss\ penalty}$$

$$= L1_{hit\ time} + L1_{miss\ rate} \times (L2_{hit\ time} + L2_{miss\ rate} \times L2_{miss\ penalty})$$

$$L1_{miss\ penalty} = L2_{hit\ time} + L2_{miss\ rate} \times L2_{miss\ penalty}$$

$$L2_{miss\ penalty} = \dots$$

$$L(n-1)_{miss\ penalty} = L_n_{hit\ time} + 0$$

Equation for AMAT

$$AMAT = L1_{hit\ time} + L1_{miss\ rate} * L1_{miss\ penalty}$$
$$= L1_{hit\ time} + L1_{miss\ rate} * (L2_{hit\ time} + L2_{miss\ rate} * L2_{miss\ penalty})$$

- * Irrespective of an hit or a miss, we need to spend some time (hit time)
- * This is the hit time in the L1 cache ($L1_{hit\ time}$)
- * This time should be discarded while calculating the stall penalty due to L1 misses
- * stall penalty = AMAT - $L1_{hit\ time}$

n-Level Memory System

$$AMAT = L1_{hit\ time} + L1_{miss\ rate} * L1_{miss\ penalty}$$

$$L1_{miss\ penalty} = L2_{hit\ time} + L2_{miss\ rate} * L2_{miss\ penalty}$$

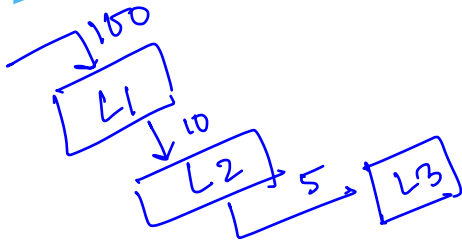
$$L2_{miss\ penalty} = L3_{hit\ time} + L3_{miss\ rate} * L3_{miss\ penalty}$$

$$\dots = \dots$$

$$L(n - 1)_{miss\ penalty} = Ln_{hit\ time}$$



Definition: Local and Global Miss Rates, Working Set



$$\textcircled{L2} = \frac{5}{10} \quad \bigg| \quad \frac{5}{150}$$

local miss rate

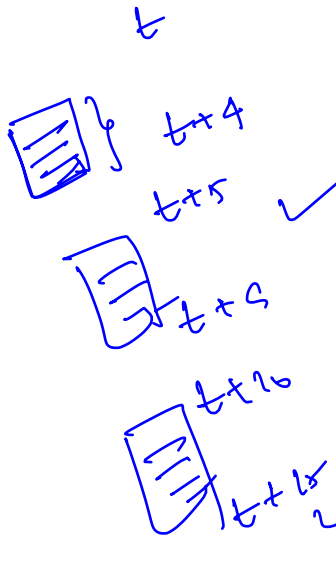
It is equal to the number of misses in a cache at level i divided by the total number of accesses at level i .

global miss rate

It is equal to the number of misses in a cache at level i divided by the total number of memory accesses.

working set

The amount of memory, a given program requires in a time interval.



Types of Misses (3c)

~ Cold start

✓ * Compulsory Misses / cold miss

- * Misses that happen when we read in a piece of data for the first time.

✓ * Conflict Misses

- * Misses that occur due to the limited amount of associativity in a set associative or direct mapped cache. **Example:** Assume that 5 blocks (accessed by the program) map to the same set in a 4-way associative cache. Only 4 out of 5 can be accommodated.

* Capacity Misses

- * Misses that occur due to the limited size of a cache. **Example:** Assume the working set of a program is 10 KB, and the cache size is 8 KB.

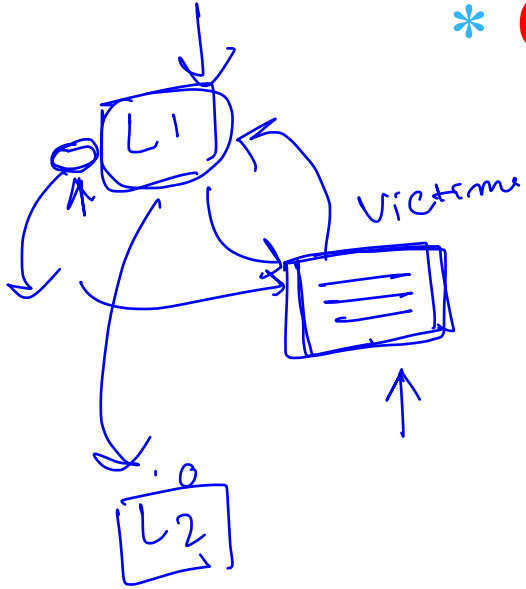
Schemes to Mitigate Misses

- * **Compulsory Misses**

- * Increase the **block size**. We can bring in more data in one go, and due to **spatial locality** the number of misses might go down.
- * Try to guess the **memory locations** that will be accessed in the near future. **Prefetch** (fetch in advance) those locations. We can do this for example in the case of array accesses.

Schemes to Mitigate Misses - II

* Conflict Misses



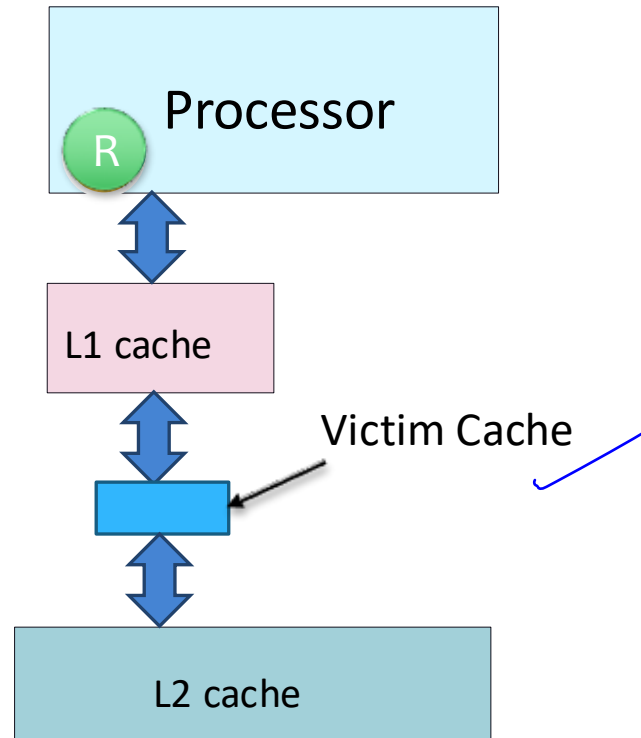
- * Increase the **associativity** of the cache (at the cost of **latency** and **power**)
- * We can use a smaller fully associative cache called the **victim cache**. Any line that gets displaced from the main cache can be put in the victim cache. The processor needs to check both the L1 and victim cache, before proceeding to the L2 cache.



* Write programs in a **cache friendly** way.

← *Compiler*

Victim Cache

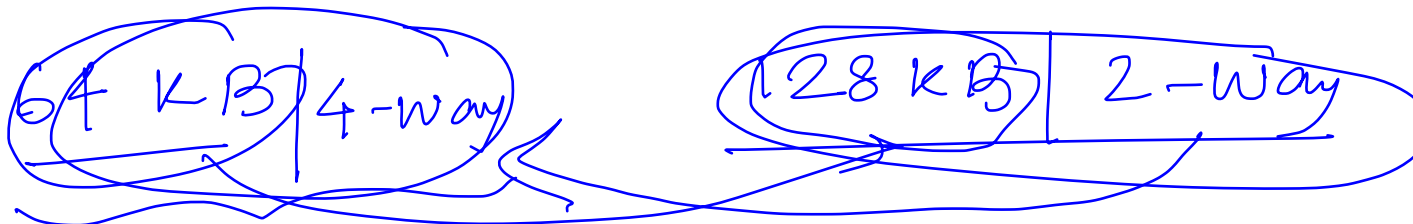


Schemes to Mitigate Misses - III

- * Capacity Misses

- * Increase the size of the cache

- * Use better prefetching techniques.



Some Thumb Rules

$$\text{miss rate} \propto \frac{1}{\sqrt{\text{cache size}}} \quad [\text{Square Root Rule}]$$

* **Associativity Rule** → Doubling the associativity is almost the same as doubling the cache size with the original associativity

* 64 KB, 4 way $\leftrightarrow$ 128 KB, 2 way

Software Prefetching

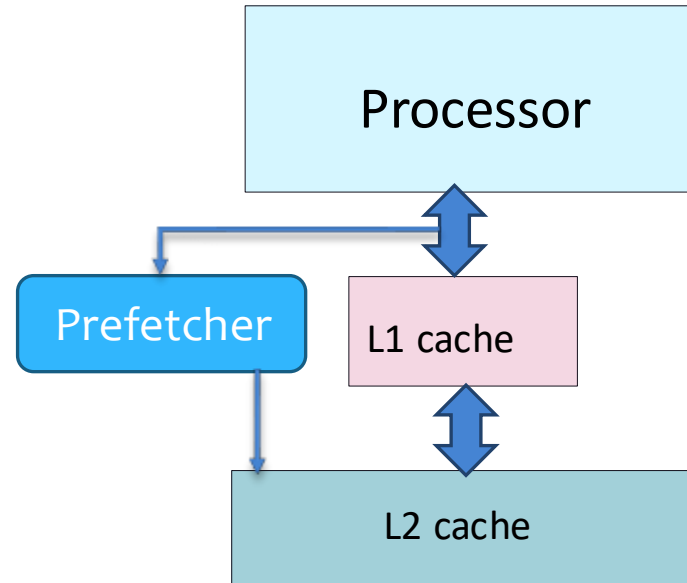
Original Code

```
int addAll(int data[], int vals[]) {  
    int i, sum = 0;  
    for (i=0; i < N; i++)  
        sum += data[vals[i]];  
    return sum;  
}
```

Modified Code with Prefetching

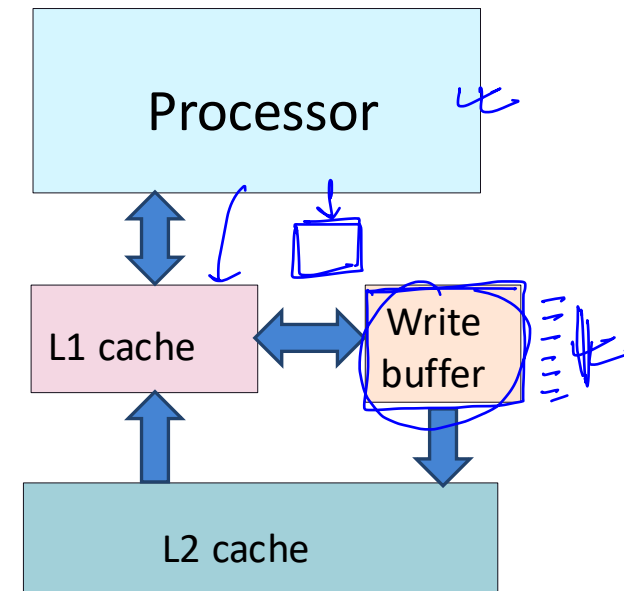
```
int addAllP(int data[], int vals[]) {  
    int i, sum = 0;  
    for (i=0; i < N; i++) {  
        __builtin_prefetch(& data[vals[i+100]] );  
        sum += data[vals[i]];  
    }  
    return sum;  
}
```

Hardware Prefetching



Reduction of Hit Time and Miss Penalty

- * For reducing the **hit time**, we need to use smaller and simpler **caches**
- * For reducing the miss penalty :
 - * **Write misses**
 - * Send the **writes** to a **fully associative** write buffer on an L1 miss.
 - * Once the **block** comes from the **L2 cache**, merge the write
 - * **Insight**: We need not send separate writes to the L2 for each write request in a block.



Reduction of the Miss Penalty

- * **Read Miss**

- * **Critical Word First** : The memory word that cause the read/write miss is fetched first from the lower level. The rest of the block follows.
- * **Early Restart** : Send the **critical word** to the processor, and make it restart its execution.

Technique	Application	Disadvantages
large block size ✓	compulsory misses ✓	reduces the number of <u>blocks</u> in the cache
prefetching //	compulsory misses, capacity misses //	extra <u>complexity</u> and the risk of displacing useful data from the cache
large cache size //	capacity misses //	high <u>latency</u> , high power, more area
increased associativity]	conflict misses //	high latency, high <u>power</u>
victim cache //	conflict misses	extra complexity ↓
compiler based techniques	all types of misses ↓	not very generic
small and simple cache ✓	hit time ✓	high miss rate ✓
write buffer ✓	miss penalty ✓	extra complexity
critical word first //	miss penalty //	extra complexity and state
early restart ✓	miss penalty	extra complexity